

```

/*
Namedle is a game inspired by Wordle.
Code is 100% by me, I didn't work with a partner.
*/

#include <bits/stdc++.h>

using namespace std;

// maximum length of name
const int maxLen = 15;

// banner displayed at the start of the program
const string banner =
"888b      888                888 888                \n\
8888b      888                888 888                \n\
88888b      888                888 888                \n\
888Y88b 888 88888b. 88888b.d88b. .d88b. .d88888 888 .d88b. \n\
888 Y88b888      \"88b 888 \"888 \"88b d8P Y8b d88\" 888 888 d8P Y8b \n\
888 Y88888 .d888888 888 888 888 88888888 888 888 888 88888888\n\
888 Y8888 888 888 888 888 888 Y8b.      Y88b 888 888 Y8b. \n\
888 Y888 \"Y888888 888 888 888 \"Y8888 \"Y88888 888 \"Y8888 \n\";

// a vector of sets that contain all possible names organized by length
vector<set<string>> names(maxLen+1);

// method that turns a string to all lowercase
string lower(string s) {
    string res = "";
    for (char c : s) {
        res += tolower(c);
    }

    return res;
}

// read names from "names" folder and put them in names vector
void initNames() {
    for (int year = 1880; year <= 2023; year++) {
        // name files are stored in names/yob_YEAR.txt, format is Name,Gender,Count
        ifstream input("names/yob"+to_string(year)+".txt");

        string line;
        while (getline(input, line)) {
            string name = "";
            for (int i = 0; i < line.size(); i++) {
                if (line[i] == ',') {
                    break;
                }
                name += line[i];
            }
            names[name.size()].insert(lower(name));
        }
        input.close();
    }
}

// check if input is a number
bool isNumber(string ans) {
    for (char c : ans) {
        if (!isdigit(c)) return false;
    }
    return true;
}

// check if input is a boolean (yes or no)
int isBool(string ans) {
    if (lower(ans) == "yes" || lower(ans) == "y") {
        return 1;
    } else if (lower(ans) == "no" || lower(ans) == "n") {
        return 0;
    }
}

```

```

    }
    return -1;
}

// generate a random name based on a given length
string generateRandomName(int len) {
    std::random_device rd;
    std::mt19937 gen(rd());
    std::uniform_int_distribution<> distr(0, names[len].size()-1);

    int rand = distr(gen);
    auto it = names[len].begin();
    for (int i = 0; i < rand; i++) {
        it++;
    }

    return lower(*it);
}

// ask the user to input the length of the name to guess
int inputLength() {
    string prompt = "Length of name to guess (2-" + to_string(maxLen) + "): ";
    cout << prompt;
    string ans;
    cin >> ans;
    int iAns;
    while (true) {
        if (!isNumber(ans)) {
            cout << "Not a number!!\n\n";
        } else {
            iAns = stoi(ans);
            if (iAns > maxLen) {
                cout << "Too long\n\n";
            } else if (iAns < 2) {
                cout << "Too short\n\n";
            } else {
                break;
            }
        }
    }

    cout << prompt;
    cin >> ans;
}

return iAns;
}

// a method to ask the user to input the number of guesses
int inputNumberOfGuesses() {
    string prompt = "Number of guesses (at least 1): ";
    cout << prompt;
    string ans;
    cin >> ans;
    int iAns;
    while (true) {
        if (!isNumber(ans)) {
            cout << "Not a number!!\n\n";
        } else {
            iAns = stoi(ans);
            if (iAns < 1) {
                cout << "Too short\n\n";
            } else {
                break;
            }
        }
    }

    cout << prompt;
    cin >> ans;
}

return iAns;
}

```

```

}

// a method that compares the guess with the answer and generates an output string
string validateAnswer(int len, string guess, string secret) {
    // add characters of guess to a frequency array
    vector<int> arr(26);
    for (int i = 0; i < len; i++) {
        arr[secret[i] - 'a']++;
    }

    // create output string
    string s(len, 'x');

    // update if the character is in the right place
    for (int i = 0; i < len; i++) {
        if (guess[i] == secret[i]) {
            arr[guess[i] - 'a']--;
            s[i] = '.';
        }
    }

    // update if the character is right but in the wrong place
    for (int i = 0; i < len; i++) {
        if (guess[i] != secret[i] && arr[guess[i] - 'a'] > 0) {
            arr[guess[i] - 'a']--;
            s[i] = 'o';
        }
    }

    return s;
}

// a method to start a round of guessing
void startRound(int len, string secret, int guesses) {
    // instructions
    cout << "\nYou have " + to_string(guesses) + " tries to guess a " + to_string(len) + "-letter name.\n\n";
    cout << "." << " = right letter, correct place" << endl;
    cout << "o = right letter, incorrect place" << endl;
    cout << "x = wrong letter\n\n";
    cout << "Type 'quit' to quit.\n\n";

    string prompt = "Your guess:\n";
    int g = guesses;
    bool won = false;

    // guessing loop
    while (g > 0) {
        cout << g << " guesses left." << endl;
        cout << prompt;

        string guess; cin >> guess; guess = lower(guess);
        if (guess == "quit") {
            break;
        } else if (guess.size() < len) {
            cout << "Too short! " << "Enter " << len << " letters.\n\n";
        } else if (guess.size() > len) {
            cout << "Too long! " << "Enter " << len << " letters.\n\n";
        } else if (names[len].find(guess) == names[len].end()) {
            cout << "Not a valid name!\n\n";
        } else { // guess is valid
            g--;
            string res = validateAnswer(len, lower(guess), secret);
            cout << res << "\n\n";
            if (res == string(len, '.')) {
                // correct answer, user won
                won = true;
                break;
            }
        }
    }
}

```

```

    if (won) {
        cout << "\nWoohoo! You guessed it in "<< guesses - g << " tries!\n\n";
    } else {
        cout << "\n0 guesses left." << endl;
        cout << "The name was " << secret << ".\n\n";
    }
}

// a method to start the game with multiple rounds until user quits
void startGame() {
    // description of game
    cout << banner << endl;
    cout << "A name guessing game" << endl;
    cout << "by Leran (Alan) Tao. Inspired by Wordle.\n\n\n";

    // add names and record the time it took
    cout << "Adding names..." << endl;
    auto t1 = chrono::high_resolution_clock::now();

    initNames();

    auto t2 = chrono::high_resolution_clock::now();
    auto ms_int = chrono::duration_cast<chrono::milliseconds>(t2 - t1);

    cout << "Done in " << ms_int.count() << "ms.\n\n";

    // main game loop
    while (true) {
        int len = inputLength();
        string secret = generateRandomName(len);
        int guesses = inputNumberOfGuesses();

        startRound(len, secret, guesses);

        // repeatedly ask for a valid boolean answer
        string ans = "";
        while (isBool(ans) == -1) {
            cout << "\nPlay again (yes/no): ";
            cin >> ans;
        }

        if (isBool(ans) == 0) {
            // user doesn't want to play again
            cout << "\nThanks for playing!" << endl;
            break;
        }

        cout << "\n\n";
    }
}

// main function to just execute startGame function when program starts
int main() {

    startGame();

    return 0;
}

```